

Trabajo Práctico Logística

**Fornari Francisco - Gareis Alejandro - Marinoni Ayrton- Pluda
Agustín - Storani Gianfranco**

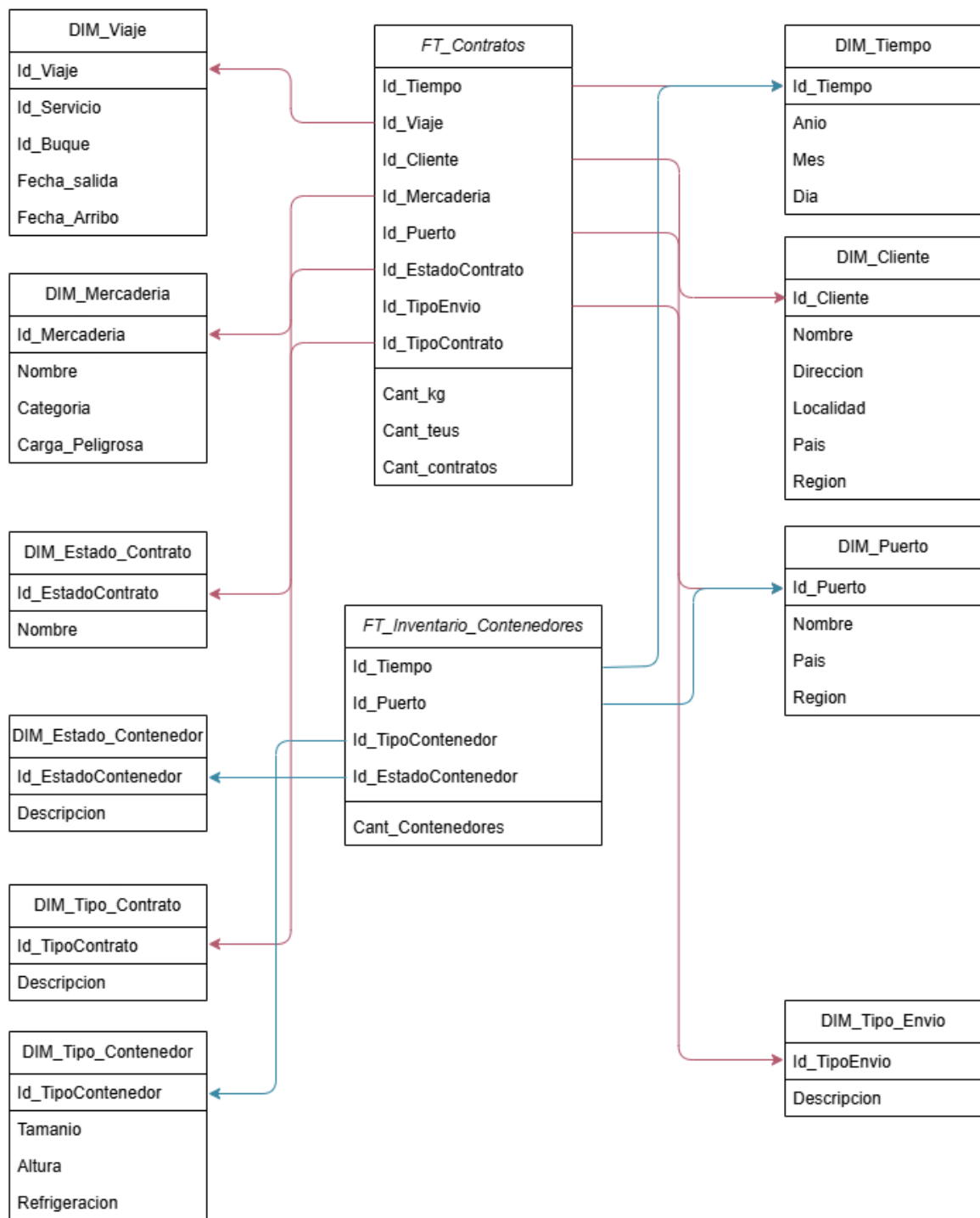


**Universidad Católica de Santiago del Estero
Departamento Académico Rafaela**

**Ingeniería en Informática
Base de Datos 3**

2025

Modelo Dimensional:



Trabajo Práctico N°1: Data Warehouse y Data Mining:

Origen de datos - Proceso ETL:

A continuación, se detalla el origen de los datos y el proceso de transformación (ETL) desde la base de datos operativa al Data Warehouse:

DIM_Tiempo: Se genera automáticamente a partir de las fechas involucradas en los contratos y movimientos de contenedores. Incluye año, mes y día.

DIM_Servicio: Se carga desde la tabla de servicios del sistema operativo, utilizando datos como buque, puerto de origen/destino y fechas.

DIM_Cliente: Se extrae de la tabla de clientes en la base de datos operativa, incluyendo dirección, localidad, país y región.

DIM_Categoria: Proviene de la clasificación de mercadería presente en los contratos.

DIM_Estado_Contrato: Se toma de los estados predefinidos de contratos en el sistema (Cancelado, En preparación, etc.).

DIM_Estado_Contenedor: Se obtiene a partir de los movimientos registrados en la tabla Movimientos_Contenedores.

DIM_Tipo_Contrato: Deriva del campo tipo de contrato (Importación/Exportación) presente en la base operativa.

DIM_Tipo_Contenedor: Se forma a partir de los datos de tamaño, altura y tipo (refrigerado o no) de los contenedores.

DIM_Puerto_Origen / DIM_Puerto_Destino: Se carga desde los puertos involucrados en los servicios y contratos.

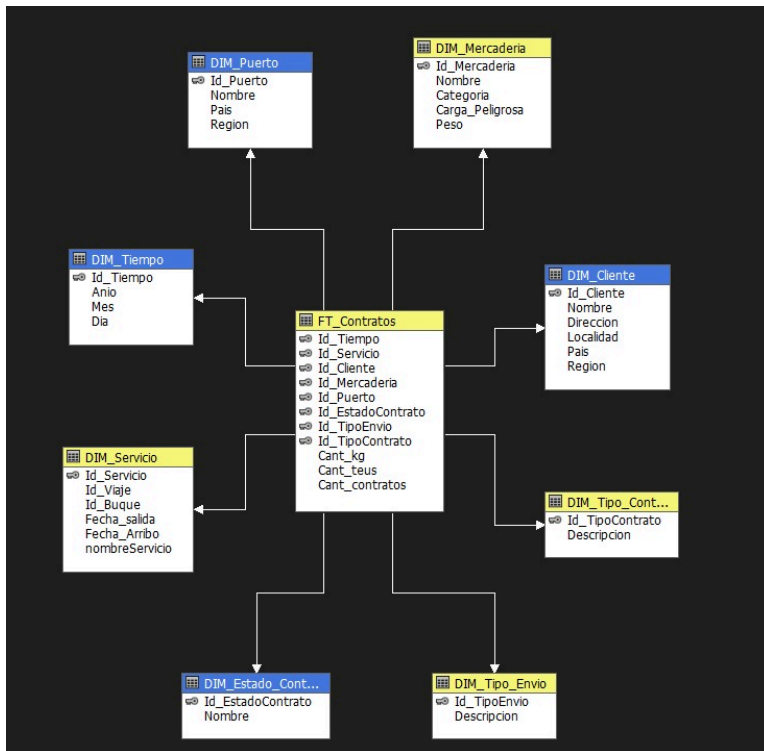
DIM_Tipo_Envio: Se extrae del tipo de envío (puerto a puerto, door to door, etc.) indicado en el contrato.

FT_Contratos: Se arma consolidando información de los contratos, cantidad de kg transportados, TEUs y cantidad de contratos. Se enlaza con dimensiones mediante los identificadores.

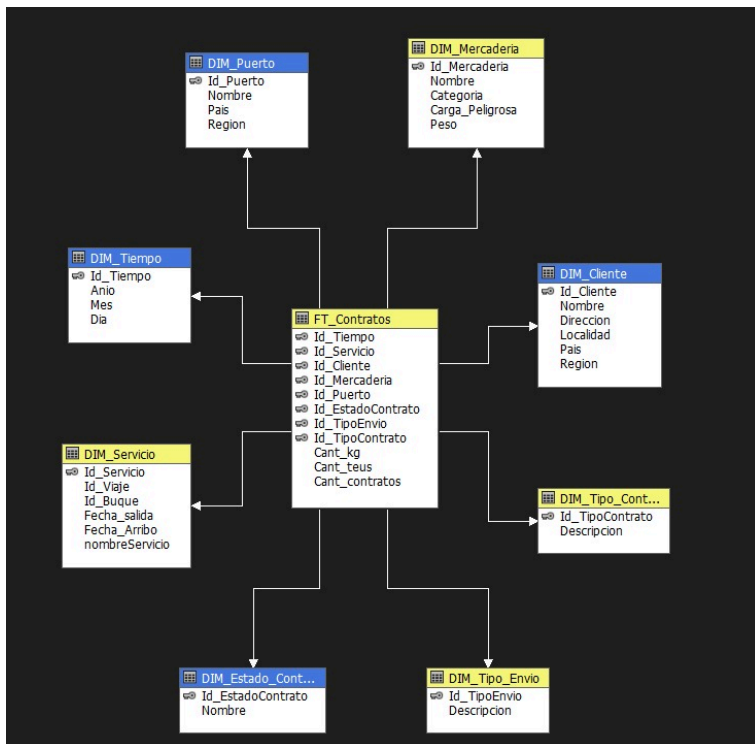
FT_Inventario_Contenedores: Se forma con base en los registros de ubicación de contenedores por puerto y fecha, teniendo en cuenta su tipo y estado. Se agrupan por día.

Estructura de los Cubos:

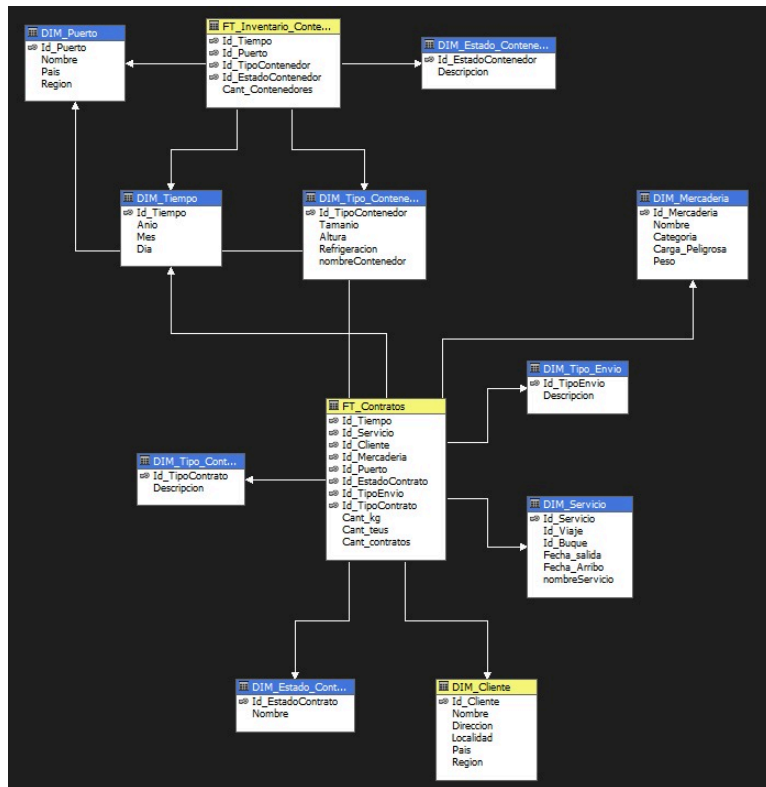
Cubo 1:



Cubo 2:



Cubo 3:



Algunas observaciones del proceso de trabajo:

Generación de los Datos:

Nos resultó difícil la generación de datos, que sean coherentes y que respeten las relaciones de las tablas.

Dificultad de uso de paquete SSIS:

Nos resultó complicado el uso de este paquete por un tema de desconocimiento sobre el mismo, pero informándonos, con el paso del tiempo pudimos realizar la actividad.

Dificultad de conexión a la base de datos:

En cada sesión que realizamos el trabajo, la base de datos se desconectaba por variados lapso de tiempo, que no nos permitía progresar.

Trabajo Práctico N°2: Jupyter Notebook:

Como dice el enunciado, utilizamos la plataforma Google Colab para poder realizar la actividad.

Notebook Completa:

Paso 1: Instalar Dependencias

Ejecutá esto en una celda para instalar los conectores necesarios:

```
[69] !pip install -q pyspark sqlalchemy sqlalchemy-pytds
```

Paso 2: Crear la conexión y cargar tablas a PySpark

Usamos SQLAlchemy con pytds, y luego lo convertimos a DataFrame de Spark:

```
from sqlalchemy import create_engine
from pyspark.sql import SparkSession

# Crear sesión de Spark
spark = SparkSession.builder.getOrCreate()

# Datos de conexión

# Conexión usando pytds (This part is not used for Spark JDBC connection)
# engine = create_engine(f"mssql+pytds://{username}:{password}@{server}/{database}")

# Función para cargar una tabla como Spark DataFrame
def cargar_tabla(nombre_tabla):
    # Modified JDBC URL to include trustServerCertificate=true to disable SSL verification (use with caution)
    jdbc_url = f"jdbc:sqlserver://{server};databaseName={database};user={username};password={password};encrypt=true;trustServerCertificate=true;"
    jdbc_driver = "com.microsoft.sqlserver.jdbc.SQLServerDriver"

    df = spark.read.format("jdbc") \
        .option("url", jdbc_url) \
        .option("dbtable", nombre_tabla) \
        .option("user", username) \
        .option("password", password) \
        .option("driver", jdbc_driver) \
        .load()

    return df
```

Paso 3: Descargar el driver JDBC y configurarlo en Colab

El conector JDBC no está por defecto en Colab, necesitás descargarlo:

```
[71] # Descargar y mover el driver JDBC para SQL Server
!wget -q https://repo1.maven.org/maven2/com/microsoft/sqlserver/mssql-jdbc/11.2.0.jre8/mssql-jdbc-11.2.0.jre8.jar -O mssql-jdbc.jar

# Configurar el driver en la sesión de Spark
spark = SparkSession.builder \
    .config("spark.jars", "mssql-jdbc.jar") \
    .getOrCreate()
```

Paso 4: Cargar las tablas necesarias

```
[72] ft_contratos = cargar_tabla("FT_Contratos")
ft_inventario_contenedores = cargar_tabla("FT_Inventario_Contenedores")
dim_tipo_contenedor = cargar_tabla("DIM_Tipo_Contenedor")
dim_tipo_envio = cargar_tabla("DIM_Tipo_Envio")
dim_tiempo = cargar_tabla("DIM_Tiempo")
dim_puerto = cargar_tabla("DIM_Puerto")
```

Paso 5: Confirmar que está todo OK

```
[97] ft_contratos.show(5)
ft_contratos.printSchema()

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Id_Tiempo|Id_Viaje|Id_Cliente|Id_Mercaderia|Id_Puerto|Id_EstadoContrato|Id_TipoEnvio|Id_TipoContrato|Cant_kg|Cant_teus|Cant_contratos|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 20250601|      1|      1|      1|      1|      1|      1|      1|      1|      1|      1|
| 20250601|      1|      1|      2|      1|      1|      1|      1|      1|      1|      1|
| 20250601|      5|      1|      1|      1|      1|      1|      1|      1|      1|      1|
| 20250601|      5|      1|      2|      1|      1|      1|      1|      1|      1|      1|
| 20250601|      9|      1|      1|      1|      1|      1|      1|      1|      1|      1|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

root
 |-- Id_Tiempo: integer (nullable = true)
 |-- Id_Viaje: integer (nullable = true)
 |-- Id_Cliente: integer (nullable = true)
 |-- Id_Mercaderia: integer (nullable = true)
 |-- Id_Puerto: integer (nullable = true)
 |-- Id_EstadoContrato: integer (nullable = true)
 |-- Id_TipoEnvio: integer (nullable = true)
 |-- Id_TipoContrato: integer (nullable = true)
 |-- Cant_kg: integer (nullable = true)
 |-- Cant_teus: integer (nullable = true)
 |-- Cant_contratos: integer (nullable = true)
```

Consulta 1: Puertos que más han participado como origen o destino de contratos

```
[98] from pyspark.sql.functions import col, count, max as spark_max

# Fecha límite (10 años)
fecha_limite = 2015

# Unir con DIM_Tiempo para filtrar los últimos 10 años
contratos_recientes = ft_contratos.join(dim_tiempo, "Id_Tiempo") \
    .filter(col("Anio") >= fecha_limite)

# Contar ocurrencias por Id_Puerto
puerto_contratos = contratos_recientes.groupBy("Id_Puerto") \
    .agg(count("*").alias("Total_contratos"))

# Obtener el/los máximos
max_puertos = puerto_contratos.select(spark_max("Total_contratos").alias("maximo"))
puertos_top = puerto_contratos.join(max_puertos, puerto_contratos["Total_contratos"] == max_puertos["maximo"])

# Agregar nombres de puerto
puertos_top.join(dim_puerto, "Id_Puerto").select("Nombre", "Total_contratos").show()

+-----+-----+
|      Nombre|Total_contratos|
+-----+-----+
|Puerto de Buenos ...|      84|
+-----+-----+
```

Consulta 2: Tipos de Contenedor que han tenido más de 500 transportes esta semana

```

1 s  from pyspark.sql.functions import lit, col, count, max as spark_max

# Semana actual (ejemplo: 1 al 30 de junio 2025)
fecha_actual = 2025
mes_actual = 6
dias_semana = list(range(1, 30))

# Filtrar por semana actual
contenedores_semana = ft_inventario_contenedores.join(dim_tiempo, "Id_Tiempo") \
    .filter((col("Anio") == fecha_actual) & (col("Mes") == mes_actual) & (col("Dia").isin(dias_semana)))

# Agrupar y filtrar
conteo_contenedores = contenedores_semana.groupBy("Id_TipoContenedor") \
    .agg(count("*").alias("Total_transportes")) \
    .filter(col("Total_transportes") > 500)

# Agregar descripción
conteo_contenedores.join(dim_tipo_contenedor, "Id_TipoContenedor").select("nombreContenedor", "Total_transportes").show()

```

nombreContenedor	Total_transportes

Consulta 2b: Tipos de Contenedor que han tenido más de 2 transportes esta semana

```

2 s  from pyspark.sql.functions import lit, col, count, max as spark_max

# Semana actual (ejemplo: 1 al 30 de junio 2025)
fecha_actual = 2025
mes_actual = 6
dias_semana = list(range(1, 30))

# Filtrar por semana actual
contenedores_semana = ft_inventario_contenedores.join(dim_tiempo, "Id_Tiempo") \
    .filter((col("Anio") == fecha_actual) & (col("Mes") == mes_actual) & (col("Dia").isin(dias_semana)))

# Agrupar y filtrar
conteo_contenedores = contenedores_semana.groupBy("Id_TipoContenedor") \
    .agg(count("*").alias("Total_transportes")) \
    .filter(col("Total_transportes") > 2)

# Agregar descripción
conteo_contenedores.join(dim_tipo_contenedor, "Id_TipoContenedor").select("nombreContenedor", "Total_transportes").show()

```

nombreContenedor	Total_transportes
refrigerado	4
no refrigerado	4
refrigerado	4
no refrigerado	3

Aclaración: Esta segunda consulta del 2 es para que se muestran los datos, porque no cargamos suficientes datos para que haya más de 500 transportes de mínimamente un tipo de contenedor.

Consulta 3: Tipos de envíos contratados en todos los meses del año


```
✓ 2 s ▶ from pyspark.sql.functions import countDistinct

# Unir con tiempo
envios_por_mes = ft_contratos.join(dim_tiempo, "Id_Tiempo")

# Contar cuántos meses únicos tiene cada tipo de envío
envios_mensuales = envios_por_mes.groupBy("Id_TipoEnvio") \
    .agg(countDistinct("Mes").alias("Meses_contratados"))

# Filtrar los que estén en los 12 meses
envios_todo_el_anio = envios_mensuales.filter(col("Meses_contratados") == 12)

# Agregar descripción
envios_todo_el_anio.join(dim_tipo_envio, "Id_TipoEnvio").select("Descripcion").show()
```

```
⇒ +-----+
  |Descripcion|
  +-----+
  +-----+
```

Consulta 3b: Tipos de envíos contratados en mas de 6 meses del año:

```
✓ 2 s ▶ from pyspark.sql.functions import countDistinct

# Unir con tiempo
envios_por_mes = ft_contratos.join(dim_tiempo, "Id_Tiempo")

# Contar cuántos meses únicos tiene cada tipo de envío
envios_mensuales = envios_por_mes.groupBy("Id_TipoEnvio") \
    .agg(countDistinct("Mes").alias("Meses_contratados"))

# Filtrar los que estén en los 12 meses
envios_todo_el_anio = envios_mensuales.filter(col("Meses_contratados") > 6)

# Agregar descripción
envios_todo_el_anio.join(dim_tipo_envio, "Id_TipoEnvio").select("Descripcion").show()
```

```
⇒ +-----+
  |Descripcion|
  +-----+
  |      Door|
  +-----+
```

Aclaración: Esta segunda consulta del 3 es para que se muestran los datos, porque no cargamos suficientes datos para que haya un tipo de envío en los 12 meses del año.

Consulta 4: Estación más fructífera (por TEUS o KG)

```
from pyspark.sql.functions import when, sum as spark_sum

# Unir con tiempo
contratos_con_tiempo = ft_contratos.join(dim_tiempo, "Id_Tiempo")

# Asignar estación
contratos_con_estacion = contratos_con_tiempo.withColumn("Estacion", when(col("Mes").isin(12, 1, 2), "Verano")
                                                         .when(col("Mes").isin(3, 4, 5), "Otoño")
                                                         .when(col("Mes").isin(6, 7, 8), "Invierno")
                                                         .when(col("Mes").isin(9, 10, 11), "Primavera"))

# Sumar TEUS por estación (o cambiar por Cant_kg si preferís)
resultado = contratos_con_estacion.groupBy("Estacion") \
    .agg(spark_sum("Cant_teus").alias("Total_teus"))

# Ordenar por mayor valor
resultado.orderBy(col("Total_teus").desc()).show()
```

Estacion	Total_teus
Invierno	224
Verano	180
Otoño	59
Primavera	59

Algunas observaciones del proceso de trabajo:

Pedido de 500 transportes en consulta 2: Son demasiados datos para pedir como consulta en una Ejercitación.

Tiempo efectivo utilizado en la resolución del TP: 31 horas aprox.